Modeling Access Control With Reflection

Document #:	P3547R1 [Latest] [Status]
Date:	2025-02-09
Project:	Programming Language C++
Audience:	EWG, LEWG, LWG

Reply-to:	Dan Katz <dkatz85@bloomberg.net> Ville Voutilainen <ville.voutilainen@gmail.com>

Contents

§ 1 Revision History

Since [P3547R0]:

— `is_accessible` fails to be constant when querying a member of a class currently under definition

— `access_context::current()` should not return a context representing a function when called from that function's parameter scope

§ 2 Abstract

We propose the addition of a `std::meta::access_context` type to [P2996R8] (“*Reflection for C++26*”), which models the “context” (i.e., enclosing namespace, class, or function) from which a class member is accessed.

An object of this type is proposed as a mandatory argument for the `std::meta::members_of` function, thereby allowing users to obtain only those class members that are accessible from some context.

A “magical” `std::meta::unchecked_access` provides an `access_context` from which all entities are accessible, thereby allowing “break glass” access for use cases that require such superpowers, while also making such uses easily audited (e.g., during code review) and clearly expressive of their intent.

A family of additional APIs more fully integrates and harmonizes the Reflection proposal with C++ access control. Notably, a `std::meta::is_accessible` function allows querying whether an entity is accessible from a context. Additional utilities are proposed to allow libraries to discover whether a class has inaccessible data members or bases without obtaining reflections of those members in the first place.

Our proposal incorporates design elements from functions first introduced by [P2996R3], and iterated on through [P2996R6], before being removed from [P2996R7] due to unresolved design elements around access to protected members (which we believe are addressed by this proposal).

The intent of this proposal it to modify P2996 such that the changes proposed here are considered concurrenty with any motion to integrate P2996 into the Working Draft.

§ 3 Background

§ 3.1 A diversity of perspectives

The discussion around whether reflection should be allowed to introspect private class members is as old as the discussion around how to bring reflection to C++. Within SG7, each such discussion has affirmed the desire to:

— obtain reflections of all members of a reflected class,

— query the properties of any reflected entity, and

— refer to reflected entities directly via the “splicing” operator, without recourse to name lookup or access control.

This design direction is incorporated into the [P2996R8] (“*Reflection for C++26*”) proposal currently targeting C++26.

— The `members_of` “metafunction” returns a `vector` containing reflections of *all* members of a class.

— Predicates like `identifier_of` and `type_of` can thereafter be used to query properties of the reflected class members.

— Splicing grants broad use of the member in many contexts (e.g., accessing the value of a static data member, forming a member access expression, obtaining a pointer to member, etc).

The topic was recently revisited at the 2024 Wrocław meeting, during which a more fragmented set of perspectives within the broader WG21 body came into focus.

— A significant contingent continues to express the position that a permissive model for class introspection is an absolute requirement for reflection.

— A nontrivial number of voices raised concerns that users may be surprised to obtain representations of private members, and that doing so ought to be possible only when accompanied by a clear expression of intent.

— Some voices pointed out that, separately from the question of whether it ought to be possible to reflect inaccessible members, it certainly ought to be possible to obtain reflections of all members to which the calling context *does* have access (i.e., “give me the acccessible members”).

— A small number of participants remain doubtful that any mechanism should be provided that allows introspection of inaccessible members.

Though it appears regrettably impossible to make all parties happy (since some perspectives listed above stand in diametric and irreconcilable opposition), we believe that progress can be made to address many of the concerns raised. In particular, we hope for this paper to address the concerns of all but the last group mentioned above.

§ 3.2 A problem of composition

Consider a simple hypothetical predicate that returns whether a class member is accessible from the calling context.

```
consteval auto is_accessible(info r) -> bool;
```

Such a function might be used as follows (using the syntax and semantics of [P2996R8]):

```
class Cls {
private:
  int priv;
};

static_assert(!is_accessible(members_of(^^Cls)[0]));
```

This looks great! But it falls apart as soon a we try to build a library on top of it.

```
consteval auto accessible_nsdms_of(info cls) -> vector<info> {
  vector<info> result;
  for (auto m : nonstatic_data_members_of(cls))
    if (is_accessible(m))
      result.push_back(m);

  return result;
}

class Cls {
private:
  int priv;
  friend void client();
```

```
};

void client() {
  static_assert(accessible_nsdms_of(^^Cls).size() > 0);  // fails
}
```

The call to `is_accessible(m)` is considered from the context of the function `accessible_nsdms_of` (which has no privileged access to `Cls`), rather than from the function `void client()` (which does).

A different model is possible, which checks access from the "point where constant evaluation begins" rather than from the point of call (i.e., as proposed by [P2996R3]). Under this model, accessibility is checked from `void client()`, since the "outermost" expression under evaluation is the *`constant-expression`* of the *`static_assert-declaration`* within that function. Although the assertion would pass under this model, other cases become strange.

```
class Cls {
  int priv;
  friend consteval int fn();
};

consteval int fn() {
  return accessible_nsdms_of(^^Cls).size();
}

int main() {
  return fn();  // returns 0
}
```

Even though `fn` is a friend of `Cls`, the constant evaluation begins in `main`: The "point of constant evaluation" model renders the friendship all but useless.

4 Proposed features

4.1 Explicit contexts

Rather than try to guess where access should be checked from, we propose that it be specified explicitly.

```
class Cls {
  int priv;
  friend consteval int fn();
};

consteval int fn() {
  return accessible_nsdms_of(^^Cls, std::meta::access_context::current()).size();
}

int main() {
  return fn();  // returns 1
}

static_assert(accessible_nsdms_of(^^Cls, std::meta::access_context::current()) == 0);
```

The `access_context::current()` function returns a `std::meta::access_context` that represents the namespace, class, or function most nearly enclosing the callsite. We propose two additional functions for obtaining an `access_context`:

— `std::meta::access_context::unprivileged()` returns an `access_context` representing unprivileged access (i.e., from the global namespace), and

— `std::meta::access_context::unchecked()` returns an `access_context` from which all entities are unconditionally accessible.

Values of type `access_context` always originate from one of these three functions. In particular, there is no mechanism for forging an `access_context` that represents an arbitrary (possibly more privileged) context.

§ 4.2 Integrating access control with `members_of`

We propose that the `std::meta::members_of` interface require a second argument which specifies an access context.

```
consteval auto members_of(info cls, access_context ctx) -> vector<info>;
```

Note that the existing permissive semantics proposed by [P2996R8] can still be achieved through use of `access_context::unchecked()`.

```
using std::meta::access_context;

class Cls {
private:
  static constexpr int priv = 42;
};

constexpr std::meta::info m = members_of(^^Cls, access_context::unchecked())[0];
static_assert(identifier_of(m) == "priv");
static_assert([:m:] == 42);
```

whereas library functions that respect access are now easy to write and to compose.

```
consteval auto constructors_of(std::meta::info cls,
                               std::meta::access_context ctx) {
  return std::vector(std::from_range,
                     members_of(cls, ctx) | std::meta::is_constructor);
}
```

Better still, writing the function once gives mechanisms for obtaining public members, accessible members, and all members. We therefore propose the removal of the `get_public_members` family of functions that was added in [P2996R7] whose sole purpose was to provide a more constrained alternative to `members_of`: With this proposal, these functions are exactly equivalent to calling `members_of` with the `access_context::unprivileged()` access context.

Perhaps unsurprisingly, we propose the same changes to `bases_of` and also propose the removal of `get_public_bases`.

§ 4.3 Modeling protected access

When accessing a protected member or base, more complex rules apply: The class whose scope the name is looked up in (i.e., the "naming class") must also be specified. For instance, given the classes

```
struct Base {
protected:
  int prot;
};

struct Derived : Base {
  void fn();
};
```

the definition of `Derived::fn` is allowed to reference the `prot`-subobject of `Derived`, i.e.,

```
this->prot = 42;
```

because the name `prot` is "looked up in the scope" of `Derived`. But it is *not* permitted to form a pointer-to-member:

```
auto mptr = &Base::prot;
```

which requires performing a search for the name `prot` in the scope of `Base`.

The inability to model these semantics is what resulted in the removal of the `access_context` API from [P2996R7]. To resolve this, we propose a member function `access_context::via(info)` that facilitates the explicit specification of a naming class.

```
void Derived::fn() {
  using std::meta::access_context;
  constexpr auto ctx1 = access_context::current();
  constexpr auto ctx2 = ctx1.via(^^Derived);

  static_assert(nonstatic_data_members_of(^^Base, ctx1).size() == 0);
    // "naming class" defaults to 'Base'; 'prot' is inaccessible as 'Base::prot'.

  static_assert(nonstatic_data_members_of(^^Base, ctx2).size() == 1);
    // OK, "naming class" is 'Derived'; 'prot' is "named via the class" 'Derived'.
}
```

With this addition, we can fully model access checking in C++. A sketch of our `access_context` class (as implemented by Bloomberg's experimental Clang/P2996 fork) is as follows:

```
class access_context {
  consteval access_context(info scope, info naming_class) noexcept
      : scope{scope}, naming_class{naming_class} { }

public:
  const info scope;  // exposition only
  const info naming_class;  // exposition only
```

```
    consteval access_context() noexcept : scope{^^::}, naming_class{} { };
    consteval access_context(const access_context &) noexcept = default;
    consteval access_context(access_context &&) noexcept = default;

    static consteval access_context current() noexcept {
      return {__metafunction(detail::__metafn_access_context), {}};
    }

    static consteval access_context unprivileged() noexcept {
      return access_context{};
    }

    static consteval access_context unchecked() noexcept {
      return access_context{{}, {}};
    }

    consteval access_context via(info cls) const {
      if (!is_class_type(cls))
        throw "naming class must be a reflection of a class type";

      return access_context{scope, cls};
    }
  };
```

§ 4.4 Additional facilities

It may be desireable for a library to determine whether a provided `access_context` is sufficiently privileged to observe the whole object representation of instances of a given class. This is easily accomplished by composing `members_of` and `bases_of` with `is_accessible`:

```
consteval auto has_inaccessible_nonstatic_data_members(info cls,
                                                       access_contxt ctx) -> bool {
  return !std::ranges::all_of(members_of(cls, std::meta::access_context::unchecked()),
                              [=](info r) { return is_accessible(r, ctx); });
}

consteval auto has_inaccessible_bases(info cls, access_contxt ctx) -> bool {
  return !std::ranges::all_of(bases_of(cls, std::meta::access_context::unchecked()),
                              [=](info r) { return is_accessible(r, ctx); });
}
```

That said, some library authors have asked for this capability to be made available through the standard library such that clients have no need to handle reflections of inaccessible members at all. We therefore propose that the above functions also be augmented to P2996.

§ 4.5 A stronger notion of access control for reflection

Previous proposals ([P3451R0], [P3473R0]) have suggested integrating access control with Reflection by checking access at splice-time. If one's goal is to prevent introspection of inaccessible member subobjects, then such a change is not enough: [P2996R8] is replete with metafunctions that, once one has a reflection, can be used to circumvent access control: `value_of`, `object_of`, `template_arguments_of`,

`extract`, `define_aggregate`, and `substitute` can all be creatively applied to circumvent access control to various extents without ever having to splice the reflection.

This proposal, therefore, presents a stronger notion of access control by making it straightforward to avoid getting reflections of inaccessible members in the first place. Like [P3451R0], we propose a "break glass" mechanism (i.e., `access_context::unchecked()`) for obtaining unconditional access to a member (e.g., to dump a debug representation of an object to `stdout`), and like that paper, the mechanism is trivially auditable (e.g., `grep unchecked_access`). The use of `unchecked_access` is deliberately **loud** and **unambiguous** in its meaning: It is very difficult to misconstrue what the author of the code intended, and hard to imagine that they will be surprised to find inaccessible members in the resulting `vector`.

Revisiting the multitude of perspectives observed within WG21, we believe this proposal should make participants with the following views happy:

— those who want a permissive model for class introspection,

— those concerned that users may be surprised to obtain representations of private members,

— those who want inaccessible members to only be returned when accompanied by a clear expression of intent, and

— those who want to obtain reflections of all accessible members, and to check whether a given member is accessible.

We believe that the union of these groups represent an overwhelming majority within WG21 and within the broader C++ community.

4.6 Implementation experience

All features proposed here are implemented by Bloomberg's experimental Clang/P2996 fork. They are enabled with the `-faccess-contexts` flag (or `-freflection-latest`).

5 Proposed wording

[Drafting note: All wording assumes the changes proposed by [P2996R8]. The following affects only the library; no core language changes are required. Our intent is to modify the text of P2996 itself such that the changes proposed here are considered concurrently with any motion to integrate P2996 into the Working Draft.]

[meta.reflection.synop] Header `<meta>` synopsis

Modify the synopsis of `<meta>` as follows:

Header `<meta>` synopsis

```
#include <initializer_list>

namespace std::meta {
```

```
    using info = decltype(^^::);

    [...]
    consteval info template_of(info r);
    consteval vector<info> template_arguments_of(info r);

    // [meta.reflection.access.context], access control context

    struct access_context {
      static consteval access_context current() noexcept;
      static consteval access_context unprivileged() noexcept;
      static consteval access_context unchecked() noexcept;

      consteval access_context via(info cls) const;

      const info scope = ^^::;  // exposition only
      const info naming_class = {};  // exposition only
    };

    // [meta.reflection.access.queries], member accessessibility queries
    consteval bool is_accessible(info r, access_context ctx);
    consteval bool has_inaccessible_nonstatic_data_members(
        info r,
        access_context ctx);
    consteval bool has_inaccessible_bases(info r, access_context ctx);

    // [meta.reflection.member.queries], reflection member queries
    consteval vector<info> members_of(info r,
                                      access_context ctx);
    consteval vector<info> bases_of(info type,
                                    access_context ctx);
    consteval vector<info> static_data_members_of(info type,
                                                  access_context ctx);
    consteval vector<info> nonstatic_data_members_of(info type,
                                                     access_context ctx);
    consteval vector<info> enumerators_of(info type_enum);

    ~~consteval vector<info> get_public_members(info type);~~
    ~~consteval vector<info> get_public_bases(info type);~~
    ~~consteval vector<info> get_public_static_data_members(info type);~~
    ~~consteval vector<info> get_public_nonstatic_data_members(info type);~~
  }
```

[meta.reflection.access.context] Access control context

Add a new subsection after [meta.reflection.queries]:

1 The `access_context` class is a structural type that represents a namespace, class, or function from which queries pertaining to access rules may be performed, as well as the naming class ([class.access.base]), if any.

```
  consteval access_context access_context::current() noexcept;
```

Let *P* be the program point at which `access_context::current()` is called.

Returns: An `access_context` whose *naming_class* is the null reflection and whose *scope* is the unique namespace, class, or function associated with the innermost namespace, class, or block scope enclosing *P*.

```
consteval access_context access_context::unprivileged() noexcept;
```

Returns: An `access_context` whose *naming_class* is the null reflection and whose *scope* is the global namespace.

```
consteval access_context access_context::unchecked() noexcept;
```

Returns: An `access_context` whose *naming_class* and *scope* are both the null reflection.

```
consteval access_context access_context::via(info cls) const;
```

Constant When: `cls` represents a class type.

Returns: An `access_context` whose *scope* is `this->`*scope* and whose *naming_class* is `cls`.

§ [meta.reflection.access.queries] Member accessibility queries

Add a new subsection after [meta.reflection.access.context]:

```
consteval bool is_accessible(info r, access_context ctx);
```

Let *P* be a program point that occurs in the definition of the entity represented by `ctx.`*scope*.

Constant When: `r` does not represent a member of a class currently being defined.

Returns:

- If `ctx.`*scope* represents the null reflection, then `true`.
- Otherwise, if `r` represents a member of a class *C*, then `true` if that class member is accessible at *P* ([class.access.base]) when named in either
  - (5.1) — `C` if `ctx.`*naming_class* is the null reflection, or
  - (5.2) — the class represented by `ctx.`*naming_classs* otherwise.
- Otherwise, if `r` represents a direct base class relationship between a base class *B* and a derived class *D*, then `true` if the base class *B* of *D* is accessible at *P*.
- Otherwise, `true`.

[*Note 1:* The definitions of when a class member or base class are accessible from a point *P* do not consider whether a declaration of that entity is reachable from *P*. — *end note*]

[*Example 1:*

```
consteval access_context fn() {
  return access_context::current();
}

class Cls {
    int mem;
    friend consteval access_context fn();
public:
    static constexpr auto r = ^^mem;
};

static_assert(is_accessible(Cls::r, fn()));  // OK
```

— *end example*]

```
consteval bool has_inaccessible_nonstatic_data_members(
      info r,
      access_context ctx);
```

8 *Returns*: `true` if `is_accessible(R, ctx)` is `false` for any `R` in `members_of(r, access_context::unchecked())`. Otherwise, `false`.

```
consteval bool has_inaccessible_bases(info r, access_context ctx);
```

9 *Returns*: `true` if `is_accessible(R, ctx)` is `false` for any `R` in `bases_of(r, access_context::unchecked())`. Otherwise, `false`.

§ [meta.reflection.member.queries] Reflection member queries

Modify the signature of `members_of` that precedes paragraph 1 as follows:

```
consteval vector<info> members_of(info r,
                                  access_context ctx);
```

Modify paragraph 4 as follows:

4 *Returns*: A `vector` containing reflections of all members-of-representable members of the entity represented by `r` that are members-of-reachable from some point in the evaluation context for which the reflection `R` satisfies `is_accessible(R, ctx)`. If `E` represents a class `C`, then the vector also contains reflections representing all unnamed bit-fields declared within the member-specification of `C` for which the reflection `R` satisfies `is_accessible(R, ctx)`. Class members and unnamed bit-fields are indexed in the order in which they are declared, but the order of namespace members is unspecified. [*Note 1:* Base classes are not members. Implicitly-declared special members appear after any user-declared members. — *end note*]

Modify the signature of `bases_of` that follows paragraph 4 as follows:

```
consteval vector<info> bases_of(info type,
                                access_context ctx);
```

Modify paragraph 6 as follows:

5 *Returns*: Let `C` be the type represented by `dealias(type)`. A `vector` containing the reflections of all the direct base class relationships, if any, of `C` for which the reflection `R` satisfies `is_accessible(R, ctx)`. The direct base class relationships are indexed in the order in which the corresponding base classes appear in the *base-specifier-list* of `C`.

Modify the signature of `static_data_members_of` that follows paragraph 6 as follows:

```
consteval vector<info> static_data_members_of(info type,
                                              access_context ctx);
```

Modify paragraph 8 as follows:

6 *Returns*: A `vector` containing each element `e` of `members_of(type, ctx)` such that `is_variable(e)` is `true`, in order.

Modify the signature of `nonstatic_data_members_of` that follows paragraph 8 as follows:

```
consteval vector<info> nonstatic_data_members_of(info type,
                                                 access_context ctx);
```

Modify paragraph 10 as follows:

10 *Returns*: A `vector` containing each element `e` of `members_of(type, ctx)` such that `is_nonstatic_data_member(e)` is `true`, in order.

Strike everything after paragraph 12 from the section:

~~`consteval vector<info> get_public_members(info type);`~~

11 ~~*Constant When:* `dealias(type)` represents a complete class type.~~

12 ~~*Returns*: A `vector` containing each element `e` of `members_of(type)` such that `is_public(e)` is `true`, in order.~~

~~`consteval vector<info> get_public_bases(info type);`~~

~~*Constant When*: `dealias(type)` represents a complete class type.~~

~~*Returns*: A `vector` containing each element `e` of `bases_of(type)` such that `is_public(e)` is `true`, in order.~~

```
consteval vector<info> get_public_static_data_members(info type);
```

~~*Constant When*: `dealias(type)` represents a complete class type.~~

~~*Returns*: A `vector` containing each element `e` of `static_data_members_of(type)` such that `is_public(e)` is `true`, in order.~~

```
consteval vector<info> get_public_nonstatic_data_members(info type);
```

~~*Constant When*: `dealias(type)` represents a complete class type.~~

~~*Returns*: A `vector` containing each element `e` of `nonstatic_data_members_of(type)` such that `is_public(e)` is `true`, in order.~~

6 References

[P2996R3] Barry Revzin, Wyatt Childers, Peter Dimov, Andrew Sutton, Faisal Vali, Daveed Vandevoorde, Dan Katz. 2024-05-22. Reflection for C++26.
https://wg21.link/p2996r3

[P2996R6] Barry Revzin, Wyatt Childers, Peter Dimov, Andrew Sutton, Faisal Vali, Daveed Vandevoorde, Dan Katz. 2024-10-10. Reflection for C++26.
https://wg21.link/p2996r6

[P2996R7] Barry Revzin, Wyatt Childers, Peter Dimov, Andrew Sutton, Faisal Vali, Daveed Vandevoorde, Dan Katz. 2024-10-13. Reflection for C++26.
https://wg21.link/p2996r7

[P2996R8] Barry Revzin, Wyatt Childers, Peter Dimov, Andrew Sutton, Faisal Vali, Daveed Vandevoorde, Dan Katz. 2024-12-16. Reflection for C++26.
https://wg21.link/p2996r8

[P3451R0] Barry Revzin. 2024-10-14. A Suggestion for Reflection Access Control.
https://wg21.link/p3451r0

[P3473R0] Steve Downey. 2024-10-16. Splicing Should Respect Access Control.
https://wg21.link/p3473r0

[P3547R0] Dan Katz, Ville Voutilainen. 2025-01-09. Modeling Access Control With Reflection.
https://wg21.link/p3547r0